

FEAST Solver Package v1.0

User Guide



Eric Polizzi
Department of Electrical and Computer Engineering,
University of Massachusetts, Amherst

References

If you are using FEAST, please cite the following publication:

- [1] E. Polizzi, *Density-Matrix-Based Algorithms for Solving Eigenvalue Problems*, Phys. Rev. B. Vol. 79, 115112 (2009)

Contact

If you have any questions or feedback regarding FEAST, please contact the author by sending an-email to **feastsolver@gmail.com**.

Acknowledgments

The author acknowledges active collaboration and interest in this work from the Intel(r) parallel and distributed solutions division (PDSD) team and the Intel(r) math kernel library (MKL) team.

The author thanks fruitful interactions and inputs from Dr. Sergey Kuznetsov, helpful discussions with Dr. Ahmed Sameh, and the support from his “solid-state” faculty colleagues at UMass, Amherst. This work has been supported by Intel PDSD and by the national science foundation (NSF) under Grant #CCF 0635196 and #ECCS 0846457.

Contents

1	Preliminary	4
1.1	The FEAST Algorithm	4
1.2	The FEAST Solver Package version v1.0	6
1.2.1	Current Features	6
1.2.2	FEAST and Dependencies	7
1.2.3	FEAST and Parallelism	7
1.3	Installation and Setup: A Step by Step Procedure	7
1.4	A simple “Hello World” Example	9
2	FEAST Interfaces	13
2.1	Basics	13
2.1.1	Definition	13
2.1.2	Common Declarations	14
2.1.3	Input FEAST parameters	14
2.1.4	Output FEAST <code>info</code> details	15
2.2	FEAST_RCI interfaces	16
2.2.1	Specific declarations	16
2.2.2	RCI mechanism	16
2.3	FEAST predefined interfaces	18
2.3.1	Specific declarations	18
2.3.2	Matrix storage	19
3	FEAST in action	20
3.1	Examples	20
3.2	FEAST utility sparse drivers	21
3.3	FEAST complements	23
3.3.1	Accuracy and convergence	23
3.3.2	Robustness of the contour integration technique	23
3.3.3	Scalability and multiplicities	24
3.3.4	Series of eigenproblems	24
3.3.5	Iterative inner system solvers	25
	FEAST: current and future directions	26

Chapter 1

Preliminary

1.1 The FEAST Algorithm

The FEAST solver package is a free high-performance numerical library for solving the standard, $\mathbf{Ax} = \lambda\mathbf{x}$, or generalized, $\mathbf{Ax} = \lambda\mathbf{Bx}$, eigenvalue problem, and obtaining all the eigenvalues λ and eigenvectors $\mathbf{x}$ within a given search interval $[\lambda_{min}, \lambda_{max}]$. It is based on an innovative fast and stable numerical algorithm presented in [1] – named the FEAST algorithm – which deviates fundamentally from the traditional Krylov subspace iteration based techniques (Arnoldi and Lanczos algorithms) or other Davidson-Jacobi techniques. The FEAST algorithm takes its inspiration from the density-matrix representation and contour integration technique in quantum mechanics. It is free from orthogonalization procedures, and its main computational tasks consist of solving very few inner independent linear systems with multiple right-hand sides and one reduced eigenvalue problem orders of magnitude smaller than the original one. The FEAST algorithm combines simplicity and efficiency and offers many important capabilities for achieving high performance, robustness, accuracy, and scalability on parallel architectures. This algorithm is expected to significantly augment numerical performances in large-scale modern applications.

Some of the capabilities of the FEAST algorithm described in [1] can be outlined as follows:

- FEAST is based on an exact mathematical derivation, the algorithm naturally captures all multiplicities;
- FEAST has the ability to re-use the basis of pre-computed subspace as suitable initial guess for performing outer-refinement iterations where high accuracy solutions are usually obtained within very few iterations ($\sim 2, 3$). This capability can also be of benefit for solving a series of eigenvalue problems that are close one another;
- the number of numerical operations is essentially independent of the size of the system and the number of eigenpairs in the search interval;
- the scalability of the FEAST algorithm is first and foremost dependent on the scalability of the inner linear system solver;
- the inner linear systems can be solved either directly or iteratively even with modest relative residual;
- FEAST exhibits important potentialities for parallelism at many different levels: (i) many search interval can be run independently, (ii) the independent inner linear systems can be solved simultaneously on each nodes (would require the distributed PFEAST package - in preparation), (iii) within a given node, the inner system solver can be parallel (current FEAST package).

Denoting N the size of the system, $[\lambda_{\min}, \lambda_{\max}]$ the search interval, N_e the number of points for the Gauss-Legendre quadrature, M_0 the size of the working subspace (where M_0 is chosen greater than the number of eigenvalues M in the search interval), the basic pseudocode for the FEAST algorithm is given in Figure 1.1 in the case of real symmetric and complex Hermitian generalized eigenvalue problems.

The performances of the basic FEAST algorithm will depend on a trade off between the choices of the number of Gauss quadrature points N_e , the size of the subspace M_0 , and the number of outer refinement loops to reach the desired accuracy. *So far, using $M_0 \geq 1.5M$, $N_e = 8$, and with at most 2 refinement loops, we have consistently obtained a relative residual equal or smaller than 10^{-10} seeking up to 1000 eigenpairs for a variety of problems.*

1- Select $M_0 > M$ random vectors $\mathbf{Y}_{N \times M_0} \in \mathbb{R}^{N \times M_0}$ 2- Set $\mathbf{Q} = 0$ with $\mathbf{Q} \in \mathbb{R}^{N \times M_0}$; $r = (\lambda_{\max} - \lambda_{\min})/2$; For $e = 1, \dots, N_e$ compute $\theta_e = -(\pi/2)(x_e - 1)$, compute $Z_e = (\lambda_{\max} + \lambda_{\min})/2 + r \exp(i\theta_e)$, solve $(Z_e \mathbf{B} - \mathbf{A})\mathbf{Q}_e = \mathbf{Y}$ to obtain $\mathbf{Q}_e \in \mathbb{C}^{N \times M_0}$ compute $\mathbf{Q} = \mathbf{Q} - (\omega_e/2)\Re\{r \exp(i\theta_e) \mathbf{Q}_e\}$ End 3- Form $\mathbf{A}_{\mathbf{Q}_{M_0 \times M_0}} = \mathbf{Q}^T \mathbf{A} \mathbf{Q}$ and $\mathbf{B}_{\mathbf{Q}_{M_0 \times M_0}} = \mathbf{Q}^T \mathbf{B} \mathbf{Q}$ rescale value of M_0 if $\mathbf{B}_{\mathbf{Q}}$ is not spd. 4- Solve $\mathbf{A}_{\mathbf{Q}} \Phi = \epsilon \mathbf{B}_{\mathbf{Q}} \Phi$ to obtain the M_0 eigenvalue ϵ_m, and eigenvectors $\Phi_{M_0 \times M_0} \in \mathbb{R}^{M_0 \times M_0}$ 5- Set $\lambda_m = \epsilon_m$ and compute $\mathbf{X}_{N \times M_0} = \mathbf{Q}_{N \times M_0} \Phi_{M_0 \times M_0}$ If $\lambda_m \in [\lambda_{\min}, \lambda_{\max}]$, λ_m is an eigenvalue solution and its eigenvector is $\mathbf{X}_m$ (the m^{th} column of $\mathbf{X}$). 6- Check convergence for the trace of the eigenvalues λ_m If iterative refinement is needed, compute $\mathbf{Y} = \mathbf{B} \mathbf{X}$ and go back to step 2	1- Select $M_0 > M$ random vectors $\mathbf{Y}_{N \times M_0} \in \mathbb{C}^{N \times M_0}$ 2- Set $\mathbf{Q} = 0$ with $\mathbf{Q} \in \mathbb{C}^{N \times M_0}$; $r = (\lambda_{\max} - \lambda_{\min})/2$; For $e = 1, \dots, N_e$ compute $\theta_e = -(\pi/2)(x_e - 1)$, compute $Z_e = (\lambda_{\max} + \lambda_{\min})/2 + r \exp(i\theta_e)$, solve $(Z_e \mathbf{B} - \mathbf{A})\mathbf{Q}_e = \mathbf{Y}$ to obtain $\mathbf{Q}_e \in \mathbb{C}^{N \times M_0}$ solve $(Z_e \mathbf{B} - \mathbf{A})^\dagger \hat{\mathbf{Q}}_e = \mathbf{Y}$ to obtain $\hat{\mathbf{Q}}_e \in \mathbb{C}^{N \times M_0}$ $\mathbf{Q} = \mathbf{Q} - (\omega_e/4)r \left(\exp(i\theta_e) \mathbf{Q}_e + \exp(-i\theta_e) \hat{\mathbf{Q}}_e \right)$ End 3- Form $\mathbf{A}_{\mathbf{Q}_{M_0 \times M_0}} = \mathbf{Q}^T \mathbf{A} \mathbf{Q}$ and $\mathbf{B}_{\mathbf{Q}_{M_0 \times M_0}} = \mathbf{Q}^T \mathbf{B} \mathbf{Q}$ rescale value of M_0 if $\mathbf{B}_{\mathbf{Q}}$ is not hpd. 4- Solve $\mathbf{A}_{\mathbf{Q}} \Phi = \epsilon \mathbf{B}_{\mathbf{Q}} \Phi$ to obtain the M_0 eigenvalue ϵ_m, and eigenvectors $\Phi_{M_0 \times M_0} \in \mathbb{C}^{M_0 \times M_0}$ 5- Set $\lambda_m = \epsilon_m$ and compute $\mathbf{X}_{N \times M_0} = \mathbf{Q}_{N \times M_0} \Phi_{M_0 \times M_0}$ If $\lambda_m \in [\lambda_{\min}, \lambda_{\max}]$, λ_m is an eigenvalue solution and its eigenvector is $\mathbf{X}_m$ (the m^{th} column of $\mathbf{X}$). 6- Check convergence for the trace of the eigenvalues λ_m If iterative refinement is needed, compute $\mathbf{Y} = \mathbf{B} \mathbf{X}$ and go back to step 2
--	--

Figure 1.1: FEAST pseudocode for solving the generalized eigenvalue problem $\mathbf{A}\mathbf{x} = \lambda\mathbf{B}\mathbf{x}$ and obtaining all the M eigenpairs within a given interval $[\lambda_{\min}, \lambda_{\max}]$. On the left, $\mathbf{A}$ is real symmetric and $\mathbf{B}$ is symmetric positive definite (spd), on the right: $\mathbf{A}$ is complex Hermitian, $\mathbf{B}$ is Hermitian positive definite (hpd). To perform the numerical integration, we have been using N_e -point Gauss-Legendre quadrature with x_e the e^{th} Gauss node associated with the weight ω_e . For example, for the case $N_e = 8$, one can use:

$$\begin{aligned}
(x_1, \omega_1) &= (0.183434642495649, 0.362683783378361), \\
(x_3, \omega_3) &= (0.525532409916328, 0.313706645877887), \\
(x_5, \omega_5) &= (0.796666477413626, 0.222381034453374), \\
(x_7, \omega_7) &= (0.960289856497536, 0.101228536290376), \text{ and } (x_{2i}, \omega_{2i})_{i=1, \dots, 4} = (-x_{2i-1}, \omega_{2i-1})
\end{aligned}$$

1.2 The FEAST Solver Package version v1.0

This general purpose FEAST solver package includes both reverse communication interfaces and ready to use predefined interfaces for dense, banded and sparse systems. It includes double and single precision arithmetic and can be called from Fortran (77,90) or C codes. FEAST is then a comprehensive library package, and yet, an easy to use software.

1.2.1 Current Features

The current version of the package is limited to solving the symmetric eigenvalue problems (real symmetric or complex Hermitian systems) on a shared-memory architecture (e.g one multicore node).

The current features of the package are summarized as follows:

- Solving $\mathbf{Ax} = \lambda\mathbf{x}$ or $\mathbf{Ax} = \lambda\mathbf{Bx}$, where $\mathbf{A}$ is real symmetric or complex Hermitian, $\mathbf{B}$ is symmetric or Hermitian positive definite
- SMP version (one node),
- Real/Complex and Single/Double precisions; *the double precision arithmetic is however highly recommended for solving large systems with high condition number.*
- Source code and pre-compiled libraries provided for common architectures (x64, ia64). *The pre-compiled libraries do not require any specific Fortran compilers (no Fortran runtime dependencies). In contrast, Fortran 90 and C compilers are both needed to compile the source code.*
- All FEAST interfaces compatible with Fortran (77 and 90) and C
- Reverse communication interfaces (RCI): **Maximum flexibility for application specific.** *Those are matrix format independent, inner system solver independent, so users must provide their own linear system solvers (direct or iterative) and mat-vec utility routines.*
- Predefined driver interfaces for dense, banded, and sparse (CSR) formats: **Less flexibility for application specific but easy ("plug and play") and optimized:**
 - FEAST_DENSE interfaces use the LAPACK library (**not included**). *The DENSE interfaces are not intended for performances in this current FEAST version.*
 - FEAST_BANDED interfaces use the SPIKE¹ banded primitives (**included**)
 - FEAST_SPARSE interfaces use the PARDISO² solver (**not included**).
- All FEAST interfaces (RCI and predefined) require the BLAS and LAPACK packages (**not included**),
- Examples and documentation included,
- FEAST utility drivers for sparse systems included: users can provide directly their sparse systems for quick testing, timing, etc. .

¹<http://software.intel.com/en-us/articles/intel-adaptive-spike-based-solver/>

²<http://www.pardiso-project.org/index.html>

1.2.2 FEAST and Dependencies

As specified above:

- the libraries LAPACK/BLAS (not provided) are needed,
- the library PARDISO (not provided) is only needed for the predefined FEAST_SPARSE interfaces.

Remark: *The Intel MKL library is highly recommended and contains both PARDISO (with also single precisions from MKL v10.2), and highly optimized LAPACK and BLAS libraries.*

1.2.3 FEAST and Parallelism

Parallelism (via shared memory programming) is not explicitly implemented in FEAST within one node i.e. the inner linear systems are currently solved one after another (avoid the fight for resources/save memory). Using the FEAST_RCI interfaces, parallelism can then be achieved if the inner system solver and the mat-vec routines provided by the user are threaded. Using the predefined FEAST interfaces, parallelism can be implicitly obtained within the shared memory version of BLAS, LAPACK or PARDISO. Also, if the PARDISO -public version- solver is used with the FEAST_SPARSE interface, the field `feastparam(10)` should be then identical to the value of the shell variable "OMP_NUM_THREADS". However, if the MKL library is used, the above field is obsolete for PARDISO and the shell variable MKL_NUM_THREADS can be used for setting automatically the number of threads (cores) for PARDISO, LAPACK and BLAS libraries. Finally, it should be noted that the PFEAST solver, in preparation, will include explicit MPI parallelism for the FEAST algorithm.

1.3 Installation and Setup: A Step by Step Procedure

In this section, we address the following question: How should you install and link the FEAST library? Please follow the following steps (for Linux/Unix systems):

1. Download the latest FEAST version in <http://www.ecs.umass.edu/~polizzi/feast>, for example, let us call this package `feast_1.0.tgz`.
2. Put the file in your preferred directory such as \$HOME directory or (for example) /opt/ directory if you have ROOT privilege.
3. Execute: `tar -xzvf feast_1.0.tgz`; Figure 1.2 represents the main FEAST tree directory being created.

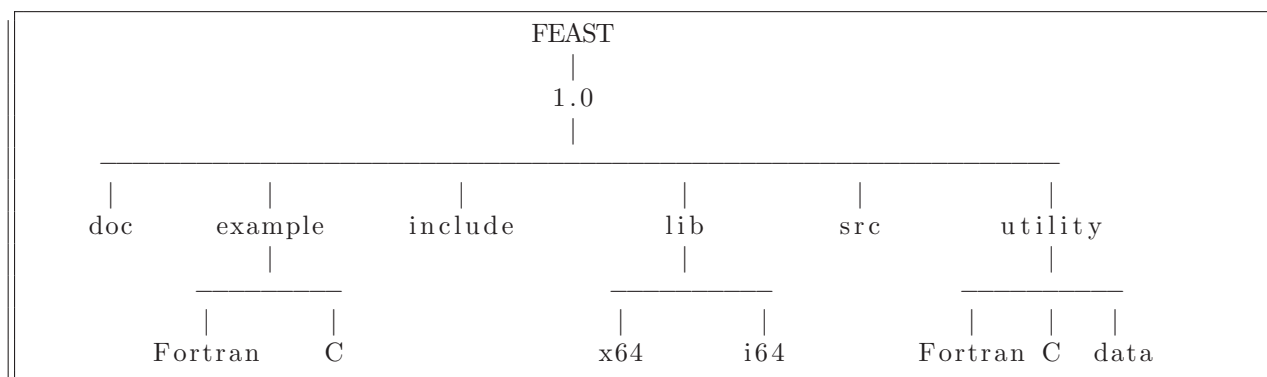


Figure 1.2: Main FEAST tree directory

4. let us denote `<FEAST directory>` the package's main directory after installation. For example, it could be

`~/home/FEAST/1.0` or `/opt/FEAST/1.0`.

It is not mandatory but recommended to define the Shell variable `$FEASTROOT`, e.g.

export `FEASTROOT=<FEAST directory>`
or **set** `FEASTROOT=<FEAST directory>`

respectively for the BASH or CSH shells. One of this command can be placed in the appropriate shell startup file in `$HOME` (i.e. `.bashrc` or `.cshrc`).

5. The FEAST pre-compiled libraries can be found at

`$FEASTROOT/lib/<arch>`

where `<arch>` denotes the computer architecture. Currently, the following architectures are provided:

- **x64** for common 64 bits architectures (e.g. Intel em64t: Nehalem, Xeon, Pentium, Centrino etc.; amd64),
- **ia64** (Intel Itanium).

It should be noted that although FEAST is written in F90, the pre-compiled libraries are free from Fortran90 runtime dependencies. If your current architecture figures in the list above, you can proceed directly to step 7, if not, you will need to compile the FEAST library in the next step.

6. *Compilation of the FEAST library source code:*

- Go to the directory `$FEASTROOT/src`
- Edit the `make.inc` file and change appropriately the name/path of the Fortran90 and C compilers (both compilers are needed to compile the source code).
- Execute: **make ARCH=<arch> all**
where `<arch>` is your selected name for your architecture; your FEAST libraries will then be created in `$FEASTROOT/lib/<arch>`.
- If you have successfully created your FEAST library for your own architecture, I will be pleased to receive these files by-email (**feastsolver@gmail.com**) in order to add them to the list of pre-compiled libraries.

7. FEAST is now installed successfully on your computer !!

In order to use the FEAST library for your F77, F90 or C application, you will then need to add the following instructions in your Makefile:

- *for the LIBRARY PATH:* `-L/$FEASTROOT/lib/<arch>`
- *for the LIBRARY LINKS: (choose one)*
 - `-lfeast_rci` (FEAST Reverse Communication Interfaces)
 - `-lfeast_dense` (FEAST dense interfaces, also includes `-lfeast_rci`)
 - `-lfeast_banded` (FEAST banded interfaces, also includes `-lfeast_rci`)
 - `-lfeast_sparse` (FEAST sparse interfaces, also includes `-lfeast_rci`)
 - `-lfeast` (FEAST library that includes all of the above)

In order to illustrate how should one use the above FEAST library links including dependencies, let us call (for example) `-lpardiso`, `-llapack`, `-lblas` respectively the library

links to the PARDISO, LAPACK and BLAS packages. The complete library links with dependencies are then given by (**choose one**):

```
-lfeast_rci -llapack -lblas
-lfeast_dense -llapack -lblas
-lfeast_banded -llapack -lblas
-lfeast_sparse -lpardiso -llapack -lblas
-lfeast -lpardiso -llapack -lblas
```

- *for the INCLUDE PATH:* `-I$(FEASTROOT)/include`

It is mandatory only for F90 codes if one chooses to make use of the instruction `-include "feast.fi"` in the header file.

1.4 A simple “Hello World” Example

This example solves a 2-by-2 dense standard eigenvalue system $\mathbf{Ax} = \lambda\mathbf{x}$ where

$$\mathbf{A} = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} \quad (1.1)$$

and the two eigenvalue solutions are known to be $\lambda_1 = 1$, $\lambda_2 = 3$ which can be associated respectively with the orthonormal eigenvectors $\pm(\sqrt{2}/2, \sqrt{2}/2)$ and $\pm(\sqrt{2}/2, -\sqrt{2}/2)$.

Let us suppose that one can specify a search interval, a single call to the `DFEAST_DST` subroutine solves then this dense standard eigenvalue system in double precision. Also, the FEAST parameters can be set to their default values by a call to the `FEASTINIT` subroutine.

The Fortran90 source code of `helloworld.f90` is listed in Figure 1.3. To create the executable, compile and link the source program with the FEAST v1.0 library, one can use:

```
ifort helloworld.f90 -o helloworld \
-L<FEAST directory>/lib/<arch> -lfeast_dense \
-lmkl_intel_lp64 -Wl,--start-group -lmkl_intel_thread -lmkl_core \
-Wl,--end-group -liomp5 -lpthread
```

where we assumed that: (i) the BLAS and LAPACK libraries are obtained via Intel MKL v10.x, (ii) the FEAST package has been installed in a directory called `<FEAST directory>`, (iii) the user architecture is `<arch>` (x64, ia64, etc.), (iv) the name of the FORTRAN compiler is “ifort”.

A run of the resulting executable looks like

```
> helloworld
```

and the output of the run appears in Figure 1.4.

Similarly, the corresponding C source code of the “helloworld” example (`helloworld.c`) is listed in figure 1.5. The executable can now be created using the “gcc” compiler (for example), along with the `-lm` library:

```
gcc helloworld.f90 -o helloworld \
-L<FEAST directory>/lib/<arch> -lfeast_dense \
-lmkl_intel_lp64 -Wl,--start-group -lmkl_intel_thread -lmkl_core \
-Wl,--end-group -liomp5 -lpthread -lm
```

```

program helloworld
  implicit none
  !! eigenvalue system
  integer ,parameter :: N=2, LDA=2
  character(len=1),parameter :: UPLO='F' ! 'L' or 'U' also fine
  double precision ,dimension(N*N) :: A=(/2.0d0,-1.0d0,-1.0d0,2.0d0/)
  double precision :: Emin=-5.0d0, Emax=5.0d0
  integer :: M0=2 ! (Initial) subspace dimension
  !! input parameters for FEAST
  integer ,dimension(64) :: feastparam
  !! output variables for FEAST
  double precision ,dimension(:),allocatable :: E, res
  double precision ,dimension(:,:),allocatable :: X
  double precision :: epsout
  integer :: i,loop,info,M

  !!! Allocate memory for eigenvalues.eigenvectors/residual
  allocate(E(M0))
  allocate(res(M0))
  allocate(X(N,M0))

  !!!!!!!!!! FEAST
  call FEASTINIT(feastparam)
  feastparam(1)=1 !! change from default value
  call DFEAST_DST(UPLO,N,A,LDA,feastparam ,epsout ,loop ,Emin,Emax,M0,&
    & E,X,M,res ,info)

  !!!!!!!!!! REPORT
  print *, 'FEAST OUTPUT INFO', info
  if (info==0) then
    print *, '*****'
    print *, '***** REPORT *****'
    print *, '*****'
    print *, '# Search interval [Emin,Emax]', Emin,Emax
    print *, '# mode found/subspace ',M,M0
    print *, '# iterations ',loop
    print *, 'TRACE',sum(E(1:M))
    print *, 'Relative error on the Trace',epsout
    print *, 'Eigenvalues/Residuals '
    do i=1,M
      print *,i,E(i),res(i)
    enddo
    print *, 'Eigenvectors '
    do i=1,M
      print *,i,"(",X(1,i),X(2,i),")"
    enddo
  endif
end program helloworld

```

Figure 1.3: A very simple F90 “helloworld” example. This code can be found in <FEAST *directory*> /example/Fortran/1_dense.

```

*****
***** FEAST- BEGIN *****
*****
Size subspace    2
#Loop | #Eig |          Trace          |          Error-trace
0      2      4.000000000000001e+00
1      2      4.000000000000000e+00    2.220446049250313e-16
==>FEAST has converged (reaches desired tolerance)
*****
***** FEAST- END*****
*****

FEAST OUTPUT INFO          0
*****
***** REPORT *****
*****
# Search interval [Emin,Emax]  -5.000000000000000    5.000000000000000
# mode found/subspace         2          2
# iterations                   1
TRACE  4.000000000000000
Relative error on the Trace  2.220446049250313E-016
Eigenvalues/Residuals
      1  1.000000000000000    1.570092458683775E-016
      2  3.000000000000000    0.000000000000000E+000
Eigenvectors
      1 (  -0.707106781186548    -0.707106781186547    )
      2 (  -0.707106781186547     0.707106781186547    )

```

Figure 1.4: Output results for the “helloworld” example.

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    /* eigenvalue system */
    int N=2,LDA=2;
    char UPLO='F'; // 'L' and 'U' also fine
    double A[2][2]={2.0,-1.0,-1.0,2.0};
    double Emin=-5.0, Emax=5.0;
    int M0=2; //size initial subspace
    /* input parameters for FEAST */
    int feastparam[64];
    /* output variables for FEAST */
    double *E, *res, *X;
    double epsout,trace;
    int i,loop,info,M;

    /* Allocate memory for eigenvalues.eigenvectors/residual */
    E=calloc(M0,sizeof(double)); //eigenvalues
    res=calloc(M0,sizeof(double)); //eigenvectors
    X=calloc(N*M0,sizeof(double)); //residual (if needed)

    /* !!!!!!!!!!! FEAST !!!!!!!!!!! */
    FEASTINIT(feastparam);
    feastparam[0]=1; /*change from default value */
    DFEAST_DST(&UPLO,&N,&A,&LDA,&feastparam,&epsout,&loop,&Emin,&Emax,&M0,\
              E,X,&M,res,&info);

    /* !!!!!!!!!!! REPORT !!!!!!!!!!! */
    printf("FEAST OUTPUT INFO %d\n",info);
    if (info==0) {
        printf("*****\n");
        printf("***** REPORT *****\n");
        printf("*****\n");
        printf("# Search interval [Emin,Emax] %.15e %.15e\n",Emin,Emax);
        printf("# mode found/subspace %d %d \n",M,M0);
        printf("# iterations %d \n",loop);
        trace=0.0;
        for (i=0;i<=M-1;i=i+1){
            trace=trace+*(E+i);
        }
        printf("TRACE %.15e\n", trace);
        printf("Relative error on the Trace %.15e\n",epsout);
        printf("Eigenvalues/Residuals\n");
        for (i=0;i<=M-1;i=i+1){
            printf(" %d %.15e %.15e\n",i,*(E+i),*(res+i));
        }
        printf("Eigenvectors\n");
        for (i=0;i<=M-1;i=i+1){
            printf(" %d (%.15e, %.15e)\n",i,*(X+i*M),*(X+1+i*M));
        }
    }
    return 0;
}

```

Figure 1.5: A very simple C “helloworld” example This code can be found in <FEAST *directory*> /example/C/1.dense.

Chapter 2

FEAST Interfaces

2.1 Basics

2.1.1 Definition

There are two different type of interfaces available in the FEAST library:

1. The reverse communication interfaces (RCI):

TFEAST_RCI

These interfaces are matrix free format (the interfaces are independent of the matrix data formats), users can then define their own explicit or implicit data format. Mat-vec routines and direct/iterative linear system solvers must also be provided by the users.

2. The predefined interfaces:

TFEAST_XYY

These interfaces can be considered as predefined optimized drivers for **TFEAST_RCI** that act on commonly used matrix data storage (dense, banded and sparse-CSR), using predefined mat-vec routines and preselected inner linear system solvers.

For theses interfaces, we note:

- **T**: the data type of matrix **A** (and matrix **B** if any) defined as follows:

Value of T	Type of matrices
S	real single precision
D	real double precision
C	complex single precision
Z	complex double precision

- **X**: the type of matrix format defined as follows:

Value of X	Matrix format	Inner linear system solver used by FEAST
D	Dense	LAPACK dense solvers
B	Banded-LAPACK	SPIKE primitives
S	Sparse-CSR	PARDISO solver

- **YY**: the eigenvalue computation – either **GE** or **ST** respectively for generalized and standard eigenvalue problem.

Remark: Using *F90* with the header - `include "feast.fi"` - all interfaces can simply be named: FEAST.

2.1.2 Common Declarations

The arguments list for the FEAST interfaces are defined as follows:

TFEAST_RCI (**{list}**, **feastparam**,**epsout**,**loop**,**Emin**,**Emax**,**M0**,**E**,**X**,**M**,**res**,**info**)

TFEAST_XYY (**{list}**, **feastparam**,**epsout**,**loop**,**Emin**,**Emax**,**M0**,**E**,**X**,**M**,**res**,**info**)

where **{list}** denotes a series of arguments that are specific to each interfaces and will be presented in the next sections. The rest of the arguments are common to both RCI and predefined FEAST interfaces and their definition is given in Table 2.1.

Component	Type (Fortran)	Input/Output	Description
feastparam	integer(64)	input	FEAST input parameters (see Table 2.2)
epsout	real <i>if T=S, C</i> or double precision <i>if T=D, Z</i>	output	Relative error on the trace
loop	integer	output	# of refinement loop executed
Emin	real <i>if T=S, C</i> or double precision <i>if T=D, Z</i>	input	Lower bound search interval
Emax	real <i>if T=S, C</i> or double precision <i>if T=D, Z</i>	input	Upper bound search interval
M0	integer	input/output	Subspace dimension On entry: initial guess (M0 > M) On exit: new suitable M0 if guess too large
E	real(M0) <i>if T=S, C</i> or double precision <i>if T=D, Z</i>	output	Eigenvalues
X	<i>Same type than T</i> <i>with 2D dimension (N,M0)</i>	input/output	Eigenvectors (N : size of the system) On entry: guess subspace if feastparam (5)=1 On exit: eigenvectors solutions
M	integer	output	# eigenvalues found between [Emin , Emax]
res	real(M0) <i>if T=S, C</i> or double precision(M0) <i>if T=D, Z</i>	output	Relative residual $\ \mathbf{Ax}_i - \lambda_i \mathbf{Bx}_i\ _1 / \ \mathbf{Ax}_i\ _1$ not referenced if feastparam (6)=0
info	integer	output	Error handling (if =0: successful exit) (see Table 2.3 for all INFO return codes)

Table 2.1: List of arguments common for the RCI and predefined FEAST interfaces.

Remark: To maximize performances users may choose **M0** 1.5 to 2 times greater than the number of eigenvalues estimated in the search interval [**Emin**,**Emax**]. In case where the initial guess **M0** is chosen too large, FEAST will automatically return a much smaller and suitable size subspace.

2.1.3 Input FEAST parameters

In the common argument list, the input parameters for the FEAST algorithm are contained into an integer array of size 64 named here **feastparam**. Prior calling the FEAST interfaces, this array can be initialized using the routine **FEASTINIT** as follows (Fortran notation):

CALL FEASTINIT(feastparam)

All input FEAST parameters are then set to their default values. The detailed list of these parameters is given in Table 2.2.

feastparam(i)	Description	Default value
i=1	Print runtime comments on screen (0: No; 1: Yes)	0
i=2	# of contour points (3,4,5,6,8,10,12,16,20,24,32,40,48)	8
i=3	Error trace- double precision stopping criteria ($10^{-\text{feastparam}(3)}$)	12
i=4	Maximum number of FEAST refinement loop allowed	20
i=5	Provide initial guess subspace (0: No; 1: Yes)	0
i=6	Compute relative residual (0: No; 1: Yes)	1
i=7	Error trace- single precision stopping criteria ($10^{-\text{feastparam}(7)}$)	5
i=10	# of threads	1
i=11	Type of algorithm (1 or 2) for TFEAST_RCI with T=C, Z 1: performs half-contour only- requires option ijob=21 2: performs full contour - does not require option ijob=21	2
i=12	Reserved value	N/A
i=20-29	Reserved values	N/A
All Others	unused	

Table 2.2: List of input FEAST parameters and default values obtained with the routine **FEASTINIT**.

Remark 1: For the case of **FEAST_SPARSE** using the public **PARDISO** version, the number of threads in **feastparam(10)** should be equal to the shell variable **OMP_NUM_THREADS**. This field is currently obsolete for the interfaces **FEAST_DENSE** and **FEAST_BANDED**. If the **MKL** library is used, however, the above field becomes also obsolete for **FEAST_SPARSE** and the shell variable **MKL_NUM_THREADS** can be used for setting the number of threads for all FEAST interfaces.

Remark 2: Using the **TFEAST_RCI** interface (with **T=C, Z**), let us suppose that solving the complex conjugate linear system $\mathbf{A}^H \mathbf{x} = \mathbf{f}$ can be achieved knowing only the factorization of **A**, the option **feastparam(11)=1** would then provide a much faster algorithm for complex Hermitian systems than its current default value “2”. The interfaces **FEAST_DENSE** and **FEAST_BANDED** are actually using the value “1”.

Remark 3: Using the C language, the components of the **feastparam** array starts at 0 and stops at 63. Therefore, the components **feastparam[j]** in C (**j=0-63**) must correspond to the components **feastparam(i)** in Fortran (**i=1-64**) specified above (i.e. **feastparam[i-1]=feastparam(i)**).

2.1.4 Output FEAST info details

Errors and warnings encountered during a run of the FEAST package are stored in an integer variable, **info**. If the value of the output **info** parameter is different than “0”, either an error or warning was encountered. The possible return values for the **info** parameter along with the error code descriptions, are given in Table 2.3.

info	Classification	Description
202	Error	Problem with size of the system N ($N \leq 0$)
201	Error	Problem with size of subspace M0 ($M0 > N$ or $M0 \leq 0$)
200	Error	Problem with Emin,Emax ($E_{min} \geq E_{max}$)
$(100 + i)$	Error	Problem with i^{th} value of the input FEAST parameter (i.e <code>feastparam(i)</code>)
3	Warning	Size of the subspace M0 is too small ($M0 \leq M$)
2	Warning	No Convergence ($\#iteration \text{ loops} > \text{feastparam}(4)$)
1	Warning	No Eigenvalue found in the search interval
0	Successful exit	
-1	Error	Internal error for allocation memory
-2	Error	Internal error of the inner system solver
-3	Error	Internal error of the reduced eigenvalue solver
		<i>Possible cause: matrix $\mathbf{B}$ may not be positive definite</i>
$-(100 + i)$	Error	Problem with the i^{th} argument of the FEAST interface

Table 2.3: Return code descriptions for the parameter `info`.

Remark: In some extreme cases the return value `info=1` may indicate that FEAST has failed to find the eigenvalues in the search interval. This situation could appear if a very large search interval is used to locate a small and isolated cluster of eigenvalues (i.e. the dimension of the search interval is many orders of magnitude off-scaling). It is then either recommended to increase the number of contour points `feastparam(2)` or simply rescale more appropriately the search interval.

2.2 FEAST_RCI interfaces

If you are not familiar with reverse communication mechanisms and/or your application does not require specific linear system solvers or matrix storage, you may want to skip this section and go directly to the section 2.3 on predefined interfaces.

2.2.1 Specific declarations

The arguments list for the FEAST_RCI interfaces is defined as follows:

`TFEAST_RCI ({list}, feastparam, epsout, loop, Emin, Emax, M0, E, X, M, res, info)`

with `{list} == {ijob, N, Ze, work1, work2, Aq, Bq}`

The definition of the series of arguments in `{list}` is given in Table 2.4.

2.2.2 RCI mechanism

The FEAST_RCI interfaces can be used to solve standard or generalized eigenvalue problems, and are independent of the format of the matrices. The reverse communication interface (RCI) mechanism is detailed in Figure 2.1. The `ijob` parameter is first initialized with the value `-1`. Once the RCI interface is called, the value of the `ijob` output parameter, if different than 0, is used to identify the FEAST operation that needs to be done. Users have then the possibility to customize their own matrix factorization and linear solve techniques as well as their own matrix multiplication routine. It should be noted that if an iterative solver is used along with a preconditioner, the factorization of the preconditioner could be performed with `ijob=10` for a given value of `Ze`, and the associated iterative solve would then be performed with `ijob=20`. Finally, for complex Hermitian system and in order to improve the performances, one may also consider the value `ijob=21` only if a complex conjugate linear system $\mathbf{A}^H \mathbf{x} = \mathbf{f}$ can be solved knowing only the factorization of $\mathbf{A}$.

Component	Type (Fortran)	Input/ Output	Description
ijob	integer	in/out	ID of the FEAST_RCI operation On entry: ijob=-1 (initialization) On exit: ijob=0,10,20,21,30,40
N	integer	in	Size of the system
Ze	complex <i>if T=S, C</i> complex*16 <i>if T=D, Z</i>	out	Coordinate along the complex contour
work1	<i>Same type than T</i> <i>with 2D dimension (N,M0)</i>	in/out	Workspace
work2	complex(N,M0) <i>if T=S, C</i> complex*16(N,M0) <i>if T=D, Z</i>	in/out	Workspace
Aq or Bq	<i>Same type than T</i> <i>with 2D dimension (M0,M0)</i>	in/out	Workspace for the reduced eigenvalue problem

Table 2.4: Definition of arguments specific for the **T**FEAST_RCI interfaces.

```

ijob=-1 ! initialization
do while (ijob/=0)
call TFEAST_RCI(ijob ,N,Ze ,work1 ,work2 ,Aq,Bq,&
               &feastparam , epsout , loop , Emin,Emax,M0,E,X,M, res , info )
  select case(ijob)
  case(10) !! Factorize the complex matrix (ZeB-A)
  .....
  case(20) !! Solve the linear system (ZeB-A)y=work2(1:N,1:M0) result in work2
  .....
  case(21) !! Solve the linear system (ZeB-A)^Hy=work2(1:N,1:M0) result in work2
             !! where ^H means transpose conjugate
             !!   ATTENTION: the case (21) is only required if:
             !!               1- T=C,Z (Complex Hermitian)
             !!               2- feastparam(11)=1
             !!   REMARK: if T=C,Z and feastparam(11)=2, case(21) is not needed
  .....
  case(30) !! Perform multiplication A*X(1:N,1:M0) result in work1(1:N,1:M0)
  .....
  case(40) !! Perform multiplication B*X(1:N,1:M0) result in work1(1:N,1:M0)
             !! (if it is a standard eigenvalue problem, just return work1=X)
  .....
  end select
end do

```

Figure 2.1: The FEAST reverse communication interface mechanism. The **T** option in **T**FEAST_RCI should be replaced by either **S**, **D**, **C** or **Z** depending of the matrix data type of the eigenvalue system.

2.3 FEAST predefined interfaces

2.3.1 Specific declarations

The arguments list for the FEAST predefined interfaces is defined as follows:

TFEAST_XYY (**{list}**, **feastparam**, **epsout**, **loop**, **Emin**, **Emax**, **M0**, **E**, **X**, **M**, **res**, **info**)

where the series of arguments in **{list}** is specific to the values of **T**, **X** and **YY**, and is given in Table 2.5. The definition of the arguments is then given in Table 2.6.

{list}	Standard: XX=ST	Generalized: XX=GE
Dense: $X=D$	$\{UPLO, N, A, LDA\}$	$\{UPLO, N, A, LDA, B, LDB\}$
Banded: $X=B$	$\{UPLO, N, k1a, A, LDA\}$	$\{UPLO, N, k1a, A, LDA, k1b, B, LDB\}$
Sparse: $X=S$	$\{UPLO, N, A, IA, JA\}$	$\{UPLO, N, A, IA, JA, B, IB, JB\}$

Table 2.5: List of arguments specific for the **TFEAST_XYY** interfaces.

Component	Type (Fortran)	Input/ Output	Description
UPLO	character(len=1)	in	Matrix Storage ('F', 'L', 'U') 'F': Full; 'L': Lower; 'U': Upper
N	integer	in	Size of the system
k1a	integer	in	The number of subdiagonals within the band of A.
k1b	integer	in	The number of subdiagonals within the band of B.
A	<i>Same type than T</i> <i>with 2D dimension: (LDA, N) if X=D</i> <i>with 2D dimension: (LDA, N) if X=B</i> <i>with 1D dimension: (IA(N+1)-1) if X=S</i>	in	Eigenvalue system (Stiffness) matrix
B	<i>Same type than T</i> <i>with 2D dimension: (LDB, N) if X=D</i> <i>with 2D dimension: (LDB, N) if X=B</i> <i>with 1D dimension: (IB(N+1)-1) if X=S</i>	in	Eigenvalue system (Mass) matrix
LDA	integer	in	Leading dimension of A; LDA>=N <i>if X=D</i> LDA>=2k1a+1 <i>if X=B; UPLO='F'</i> LDA>=k1a+1 <i>if X=B; UPLO/='F'</i>
LDB	integer	in	Leading dimension of B; LDB>=N <i>if X=D</i> LDB>=2k1b+1 <i>if X=B; UPLO=='F'</i> LDB>=k1b+1 <i>if X=B; UPLO/='F'</i>
IA	integer(N+1)	in	Sparse CSR Row array of A.
JA	integer(IA(N+1)-1)	in	Sparse CSR Column array of A.
IB	integer(N+1)	in	Sparse CSR Row array of B.
JB	integer(IB(N+1)-1)	in	Sparse CSR Column array of B.

Table 2.6: Definition of arguments specific for the **TFEAST_XYY** interfaces.

2.3.2 Matrix storage

Let us consider a standard eigenvalue problem and the following (stiffness) matrix $\mathbf{A}$:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & 0 & 0 \\ a_{21} & a_{22} & a_{23} & 0 \\ 0 & a_{32} & a_{33} & a_{34} \\ 0 & 0 & a_{43} & a_{44} \end{pmatrix} \quad (2.1)$$

where $a_{ij} = a_{ji}^*$ for $i \neq j$ (i.e. $a_{ij} = a_{ji}$ if the matrix is real). Using the FEAST predefined interfaces, this matrix could be stored in dense, banded or sparse format as follows:

- Using the dense format, $\mathbf{A}$ is stored in a two dimensional array in a straightforward fashion. Using the options `UPL0='L'` or `UPL0='U'`, the lower triangular and upper triangular part respectively, do not need to be referenced.
- Using the banded format, $\mathbf{A}$ is also stored in a two dimensional array following the banded LAPACK-type storage:

$$\mathbf{A} = \begin{pmatrix} * & a_{12} & a_{23} & a_{34} \\ a_{11} & a_{22} & a_{33} & a_{44} \\ a_{21} & a_{32} & a_{43} & * \end{pmatrix}$$

In contrast to LAPACK, no extra-storage space is necessary since `LDA` $\geq 2 * \text{kla} + 1$ if `UPL0='F'` (LAPACK banded storage would require `LDA` $\geq 3 * \text{kla} + 1$). For this example, the number of subdiagonals or superdiagonals is `kla`=1. Using the option `UPL0='L'` or `UPL0='U'`, the `kla` rows respectively above or below the diagonal elements row, do not need to be referenced (or stored).

- Using the sparse storage, the non-zero elements of $\mathbf{A}$ are stored using a set of one dimensional arrays (`A`, `IA`, `JA`) following the definition of the CSR (Compressed Sparse Row) format

$$\begin{aligned} \mathbf{A} &= (a_{11}, a_{12}, a_{21}, a_{22}, a_{23}, a_{32}, a_{33}, a_{34}, a_{43}, a_{44}) \\ \mathbf{IA} &= (1, 3, 6, 9, 11) \\ \mathbf{JA} &= (1, 2, 1, 2, 3, 2, 3, 4, 3, 4) \end{aligned}$$

Using the option `UPL0='L'` or `UPL0='U'`, one would get respectively

$$\begin{aligned} \mathbf{A} &= (a_{11}, a_{21}, a_{22}, a_{32}, a_{33}, a_{43}, a_{44}) & \mathbf{A} &= (a_{11}, a_{12}, a_{22}, a_{23}, a_{33}, a_{34}, a_{44}) \\ \mathbf{IA} &= (1, 2, 4, 6, 8) & \mathbf{IA} &= (1, 3, 5, 7, 8) \\ \mathbf{JA} &= (1, 1, 2, 2, 3, 3, 4) & \mathbf{JA} &= (1, 2, 2, 3, 3, 4, 4) \end{aligned}$$

Finally, the (mass) matrix $\mathbf{B}$ that appears in generalized eigenvalue systems, should use the same family of storage format than the matrix $\mathbf{A}$. It should noted, however, that the bandwidth can be different for the banded format (`k1b` can be different than `k1a`), and the position of the non-zero elements can also be different for the sparse format (CSR coordinates `IB`, `JB` can be different than `IA`, `JA`).

Chapter 3

FEAST in action

3.1 Examples

The `$FEASTROOT/example` directory provides Fortran and C examples for using the FEAST pre-defined interfaces. The Fortran examples are written in F90 but it should be straightforward for the user to transform them into F77 if needed (since FEAST uses F77 argument-type interfaces). Examples are limited at solving two particular types of eigenvalue problems:

Example 1 a “real symmetric” generalized eigenvalue problem $\mathbf{Ax} = \lambda\mathbf{Bx}$, where $\mathbf{A}$ is real symmetric and $\mathbf{B}$ is symmetric positive definite. $\mathbf{A}$ and $\mathbf{B}$ are of the size $N = 1671$ and have the same sparsity pattern with number of non-zero elements $NNZ = 11435$.

Example 2 a “complex Hermitian” standard eigenvalue problem $\mathbf{Ax} = \lambda\mathbf{x}$, where $\mathbf{A}$ is complex Hermitian. $\mathbf{A}$ is of size $N = 600$ with number of non-zero elements $NNZ = 2988$.

Each problem is solved while considering either single or double precision arithmetic and either a dense, banded or sparse storage for the matrices. The directories `$FEASTROOT/example/Fortran` and `$FEASTROOT/example/C`, contain both the subdirectories `1_dense`, `2_banded`, `3_sparse` with source code drivers for the above two examples in single and double precisions.

In order to compile and run the examples of the FEAST package, please follow the following steps:

1. Go to the directory `$FEASTROOT/example`
2. Edit the `make.inc` file, and customize appropriately the name/path of your F90 and C compilers, the path `LOCLIBS` and names in `LIBS` of the PARDISO, LAPACK, and BLAS libraries. The path to the FEAST libraries needs also to be specified in `LOCLIBS`. If you wish to compile and run the F90 examples alone, it is not necessary to specify the C compiler, and vice-versa. If you do not wish to compile the sparse examples there is no need to specify the path and name of the PARDISO library. By default, `LIBS` in `make.inc` uses the Intel-MKL v10.x to link the required libraries.
3. By executing `make`, all the different Makefile options will be printed out including compiling alone, compiling and running, and cleaning. For example,

```
>make allF
```

compiles all Fortran examples, while

```
>make rallF
```

compiles and then run all Fortran examples.

4. If you wish to compile and/or run a given example, for a particular language and with a particular storage format, just go to one the corresponding subdirectories `1_dense`, `2_banded`, `3_sparse` of the directories `$FEASTROOT/example/Fortran` or `$FEASTROOT/example/C`. You will find a local `Makefile` (using the options defined above in the `make.inc` file) as well as the four source codes covering the **example 1** and **example 2** above in single and double precisions. The `1_dense` directories include also the “helloworld” source code examples presented in Section 1.4. Denoting **x** either **f90** or **c** depending on the Fortran or C example directory, we get that:

- The `1_dense` subdirectory contains:
 - `helloworld.x` (solve the helloworld example presented in the documentation)
 - `{s,d}example_dense1.x` (solve example1 using `{S,D}`FEAST_DGE interfaces)
 - `{c,z}example_dense2.x` (solve example2 using `{C,Z}`FEAST_DST interfaces)
- The `2_banded` subdirectory contains:
 - `{s,d}example_banded1.x` (solve example1 using `{S,D}`FEAST_BGE interfaces)
 - `{c,z}example_banded2.x` (solve example2 using `{C,Z}`FEAST_BST interfaces)
- The `3_sparse` subdirectory contains:
 - `{s,d}example_sparse1.x` (solve example1 using `{S,D}`FEAST_SGE interfaces)
 - `{c,z}example_sparse2.x` (solve example2 using `{C,Z}`FEAST_SST interfaces)

3.2 FEAST utility sparse drivers

If a sparse matrix can be provided by the user in coordinate format, the `$FEASTROOT/utility` directory offers a quick way to test the efficiency/reliability of the FEAST SPARSE predefined interfaces using the PARDISO solver. A general driver is provided for Fortran and C, named `driver_feast_sparse` and respectively located in the subdirectories `$FEASTROOT/utility/Fortran` and `$FEASTROOT/utility/C`. You will also find a local `Makefile` where compiler and libraries paths/names need to be edited and changed appropriately (the PARDISO solver is needed). The command “`>make all`” should compile the Fortran or C driver.

Let us now denote `mytest` a generic name for the user’s eigenvalue system test $\mathbf{Ax} = \lambda\mathbf{x}$ or $\mathbf{Ax} = \lambda\mathbf{Bx}$. You will need to create the following three files:

- `mytest.A` shall contain the matrix **A** in coordinate format; As a reminder, the coordinate format is defined row by row as

N	N	NNz
i1	j1	val1
i2	j2	val2
i3	j3	val3
.	.	.
.	.	.
iNNZ	jNNZ	valNNZ

with N: size of matrix, and NNZ: number of non-zero elements.

- `mytest.B` shall contain the matrix **B** in coordinate format (if any);
- `mytest.in` shall contain the search interval, some selected FEAST parameters, etc. The following `.in` file is given as a template example for solving a standard eigenvalue problem in double precision:

```

s      ! "s"tandard or "g"eneralized eigenvalue problem
d      !(s,d,c,z)precision i.e (single real,double real,complex,double complex)
F      ! UPLO (matrix elements provided:'F' Full,'L' Lower part,'U' upper part)
-5.0e0 ! Emin (lower bound search interval)
5.0e0  ! Emax (upper bound search interval)
20     ! M0 (size of the subspace)
!!!!FEASTPARAM (1,64)) in Fortran; [0,63] in C
1      !feastparam(1)[0] !(0,1)
8      !feastparam(2)[1] !(3,4,5,6,8,10,12,16,20,24,32,40,48)
12     !feastparam(3)[2] if (s,c) precision , or feastparam(7)[6] if (d,z) precision
20     !feastparam(4)[3] !maximum #loop
1      !feastparam(6)[5] !(0,1)
1      !feastparam(10)[9]!(number of threads for PARDISO public version)

```

You may change any of the above options to fit your needs. It should be noted that the UPLO L or U options give you the possibility to provide only the lower or upper triangular part of the matrices `mytest.A` and `mytest.B` in coordinate format.

Finally results and timing can be obtained by running:

```
>$FEASTROOT/utility/Fortran/driver_feast_sparse <PATH_TO_MYTEST>/mytest
```

or

```
>$FEASTROOT/utility/C/driver_feast_sparse <PATH_TO_MYTEST>/mytest
```

respectively for the Fortran or C drivers.

In order to illustrate a direct use of the utility drivers, four examples are provided in the directory `$FEASTROOT/utility/data` with the following generic name:

- `system1` (identical to example 1 presented in section 3.1),
- `system2` (identical to example 2 presented in section 3.1),
- `helloworld` (helloworld example provided in this documentation, now in sparse format),
- `cnt` (a realistic example from Carbon Nanotube Nanoelectronics Simulation used in [1]),

To run a specific test, you can execute (using the Fortran driver for example):

```
>$FEASTROOT/utility/Fortran/driver_feast_sparse ../data/cnt
```

3.3 FEAST complements

This section points out the potentialities and features of the FEAST package in terms of numerical stability, robustness and scalability. More detailed information on the FEAST algorithm and simulation results with comparisons with the ARPACK package are provided in [1].

3.3.1 Accuracy and convergence

The performances of FEAST can depend on some tunings parameters such as the choices of the size of the subspace `M0`, the number of contour points `feastparam(2)`, and the stopping criteria for the error on the trace (`feastparam(3)` for double precision or `feastparam(7)` for single precision).

In Table 3.1, we report the times and relative residual obtained by the FEAST_SPARSE interface for solving the `cnt` example provided in the directory `$FEASTROOT/utility/data`, and seeking up to `M=800` eigenpairs in a given search interval. The simulation runs are here restricted to a given node of a 8-cores Intel Clovertown system (16Gb, 2.66GHz). In our experiments, the convergence criteria on the relative residual for FEAST is obtained when the relative error on the trace of the eigenvalues in the search interval is smaller or equal to 10^{-13} (i.e. `feastparam(3)=13`). Table 3.2 shows the variation of the relative error on the trace (`epsout`) with the number of outer-iterative refinement for FEAST (`loop`). In these experiments, only 2 to 3 refinement loops suffice to obtain the small relative residuals for the different cases reported in Table 3.1. It should be noted that with only one loop, eigenvalues are obtained with an accuracy of $\sim 10^{-5}$ or below.

cnt N=12,450	FEAST	
	Time(s)	Resid.
M=100	7.8	$4.5 * 10^{-10}$
M=200	14	$5.5 * 10^{-10}$
M=400	21	$1.8 * 10^{-10}$
M=800	58	$3.4 * 10^{-11}$

Table 3.1: FEAST timing with relative residuals $\max_i(\|\mathbf{Ax}_i - \lambda_i \mathbf{Bx}_i\|_1 / \|\mathbf{Ax}_i\|_1)$, obtained for different search intervals. The size of the subspace has been chosen to be `M0=1.5M`, and the number of contour points for FEAST is `feastparam(2) = 8`.

cnt N=12,450	Relative error on the Trace		
	1st loop	2nd loop	3rd loop
M=100	$3.0 * 10^{-6}$	$2.9 * 10^{-12}$	$1.0 * 10^{-15}$
M=200	$1.8 * 10^{-5}$	$4.8 * 10^{-12}$	$2.1 * 10^{-14}$
M=400	$2.4 * 10^{-8}$	$3.2 * 10^{-16}$	
M=800	$1.8 * 10^{-9}$	$4.3 * 10^{-16}$	

Table 3.2: Relative error on the trace `epsout` in function of the number of outer-iterative refinement `loop`. The convergence criteria is set to `feastparam(3) = 13` where the final relative residual on the eigenpairs is reported in Table 3.1.

3.3.2 Robustness of the contour integration technique

The simulation results in Table 3.1 demonstrate very good numerical stability for FEAST while the search interval keeps increasing and the number of contour points `feastparam(2)=8` stays identical (i.e. the number of numerical operations stays the same for a given loop of FEAST with a fixed number of linear systems to solve). In addition, from Table 3.3, one can see how the robustness of the FEAST algorithm is affected while the number of contour points `feastparam(2)` changes. In comparisons to the results reported in Table 3.1, 4 contour points did suffice to capture `M=100` eigenpairs with a relatively small residual and a faster simulation time, while the case of 16 contour points generated a smaller relative residual. One could also show that for a fixed number of contour point, increasing the size of subspace `M0` should also improve the convergence.

As a general rule, better accuracy (`epsout`) and better convergence (i.e. small value for `loop`) can be obtained while increasing the number of contour points `feastparam(2)`. However, the simulation time would also increase since more linear systems need to be solved by `loop`. Therefore, the fastest time can be obtained while considering a trade off between the choices of `feastparam(2)`, `M0` and the desired accuracy `feastparam(3)` (for double precision).

cnt M = 100	FEAST		
	Time(s)	Resid.	# loops
N _e = 4	7.0	$8.3 * 10^{-8}$	6
N _e = 8	7.8	$4.5 * 10^{-10}$	4
N _e = 16	10.2	$3.4 * 10^{-12}$	3

Table 3.3: Performance results obtained by FEAST seeking $M = 100$ eigenpairs for different values of `feastparam(2)`. The convergence is obtained when the error on the trace `epsout` is equal or smaller to 10^{-13} .

3.3.3 Scalability and multiplicities

We propose to create artificially new `cnt` systems called $k(N, M)$ where the matrices $\mathbf{A}$ and $\mathbf{B}$ are repeated k times along the main diagonal (the new system matrix is block diagonal with k blocks). If we keep the same search interval used to obtain $M=100$ eigenpairs for $k=1$ (where the size of the matrices $\mathbf{A}$ and $\mathbf{B}$ is N), $100k$ eigenpairs must now be found for $k \geq 1$, where each one of them have the multiplicity k . In Table 3.4, we report the simulation times and relative residuals obtained using FEAST on these $k(N, M)$ `cnt` systems. For the case $8(N, M)$, for example, the size of the new system matrix is 99,600 and the first 100 eigenvalues have all the multiplicity 8 (so 800 eigenpairs are found in total). FEAST captures all multiplicities while the number of operations stays the same for all these cases (i.e. `feastparam(2)` is fixed at 8). The simulation results show linear scalability performances with the size of the system and the number of eigenpairs (i.e. numerical complexity of $O(NM)$). In practice, the scalability of the FEAST will depend mainly on the scalability of the inner linear system solver.

cnt N = 12,450 M = 100	FEAST	
	Time(s)	Resid.
(N, M)	7.8	$4.5 * 10^{-10}$
2(N, M)	27	$7.7 * 10^{-10}$
4(N, M)	109	$8.8 * 10^{-10}$
8(N, M)	523	$6.5 * 10^{-10}$

Table 3.4: Simulations times and relative residual obtained using the FEAST_SPARSE interface on the $k(N, M)$ `cnt` systems which artificially reproduce k times the original `cnt` system. kM eigenpairs are found where each eigenvalue has a multiplicity of k .

3.3.4 Series of eigenproblems

We know that FEAST can re-use the computed subspace as suitable initial guess for performing iterative refinements. This capability can also be of benefit to modern applications in science and engineering where it is often necessary to solve a series of eigenvalue problems that are close one another. For example, let us consider many eigenvalue problems of the form $\mathbf{A}_k \mathbf{x}_k = \lambda_k \mathbf{B}_k \mathbf{x}_k$ that need to be solved for different values of k . Using the FEAST package, after the first k point, the input parameter `feastparam(5)` could take the value 1, and then the 2D input array $\mathbf{X}$ would contain the eigenvectors obtained at the point $k - 1$. Figure 3.1 illustrates a problem-type using the same search interval for the eigenvalues λ for different values of k where the subspace computed by FEAST at the point $k - 1$ is successively used as initial guess for the neighboring point k .

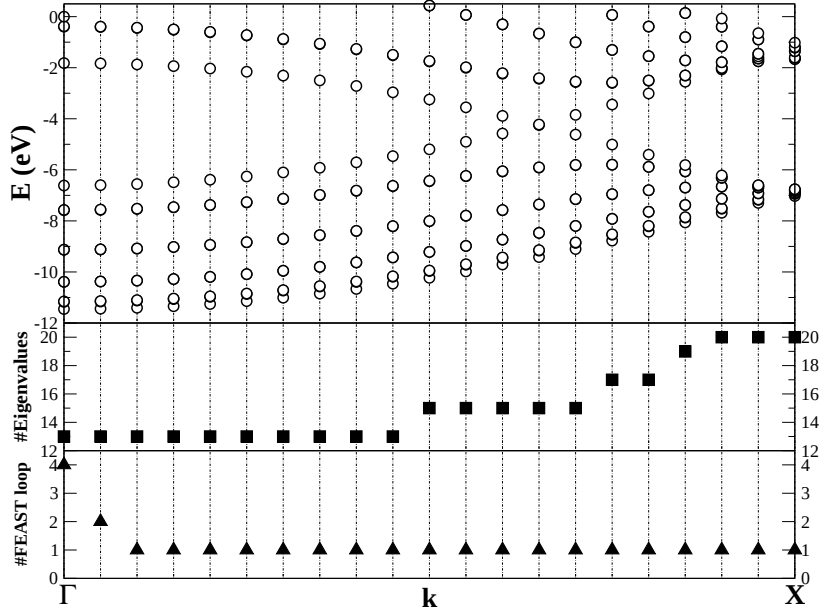


Figure 3.1: Bandstructure calculations of a (5,5) metallic carbon nanotube (CNT). The eigenvalue problems are solved successively for all the $\mathbf{k}$ points (from Γ to $\mathbf{X}$), while the computed subspace of size $M_0=25$ at the point $\mathbf{k}$ is used as initial guess for the point $\mathbf{k} + 1$. The number of eigenvalues found ranges from 13 to 20, and by the third $\mathbf{k}$ point, the FEAST convergence is obtained using only one refinement loop.

3.3.5 Iterative inner system solvers

Using the FEAST algorithm, the difficulty of solving an eigenvalue problem has been replaced by the difficulty of solving a linear system with multiple right-hand sides. For large sparse systems, this latter can be solved using either a direct system solver such as PARDISO (as predefined in the FEAST_SPARSE interface), or an iterative system solver with preconditioner (that can be customized by the user within the FEAST_RCI interface). It should be noted that the inner linear systems arising from standard eigenvalue solvers (using the shift-strategy), need often to be solved highly accurately via direct methods. Direct system solvers, however, are not always suited for addressing large-scale modern applications because of memory requirements.

In Figure 3.1, the inner linear systems in FEAST are actually solved using an iterative method with preconditioner where a modest relative residual of 10^{-3} is used. The convergence criteria for the relative error on the trace of the eigenvalues is chosen much smaller at 10^{-8} (`feastparam(3)=8`), while the eigenvectors are expected to be obtained within the same (or a smaller) order of accuracy that the one used for the solutions of the inner systems (in the simulation results, however, the final relative residuals on the eigenpairs range from 10^{-3} to 10^{-5}). Finally, the resulting subspace could also be used as a very good initial guess for a one step more accurate refinement procedure (i.e. using more accurate relative residual for the inner linear systems).

FEAST: current and future directions

Three library projects are currently in preparation:

- FEAST extended to non-symmetric eigenvalue problems where the contour integration would then be performed in a given region of the complex space;
- a SPIKE-openMP package to solve the banded linear systems on shared memory architectures and would increase the scalability of the FEAST_BANDED interface;
- A PFEAST solver package that will include explicit MPI parallelism for the FEAST algorithm and then enable large scale cluster computing. Given the recent advances in parallel architectures and parallel linear system solvers, it is reasonable to envision using the PFEAST package in a near future for obtaining up to millions of eigenpairs of large sparse eigenvalue problems.